Type your text

September 21, 2025

Q1 *A)*

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

[3]:
```
data=pd.read_csv(r"C:DATA\iris.csv")
```

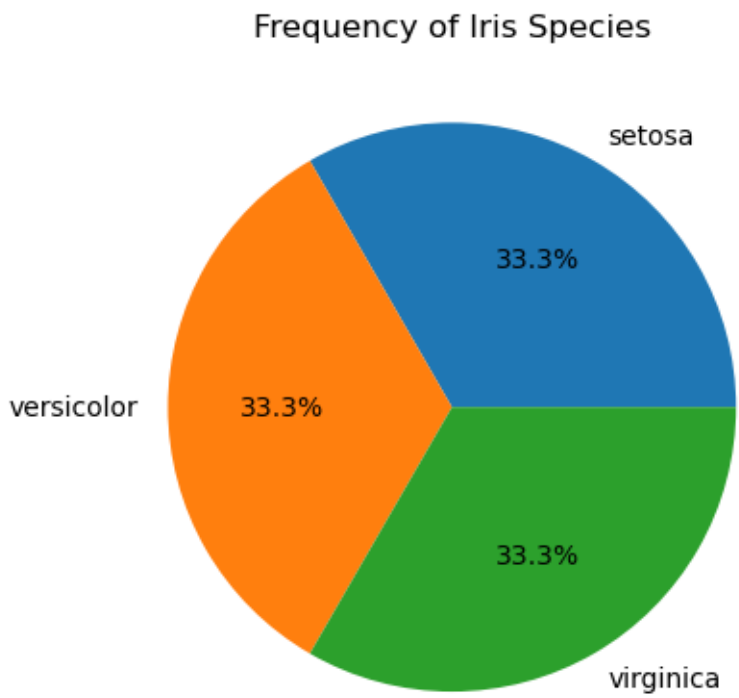
[4]:
```
data
```

[4]:
```
     Sepal.Length  Sepal.Width  Petal.Length  Petal.Width    Species
0             5.1          3.5           1.4          0.2     setosa
1             4.9          3.0           1.4          0.2     setosa
2             4.7          3.2           1.3          0.2     setosa
3             4.6          3.1           1.5          0.2     setosa
4             5.0          3.6           1.4          0.2     setosa
..            …            …             …            …         …
145           6.7          3.0           5.2          2.3  virginica
146           6.3          2.5           5.0          1.9  virginica
147           6.5          3.0           5.2          2.0  virginica
148           6.2          3.4           5.4          2.3  virginica
149           5.9          3.0           5.1          1.8  virginica

[150 rows x 5 columns]
```

[5]:
```
species_counts = data['Species'].value_counts()
```

[6]:
```
plt.pie(species_counts, labels=species_counts.index,autopct='%1.1f%%')
plt.title('Frequency of Iris Species')
plt.show()
```



B)

[7]:
```
data=pd.read_csv(r"C:DATA\winequality-red.csv")
```

[8]:
```
data
```

[8]:
```
      fixed acidity  volatile acidity  citric acid  residual sugar  chlorides  \
0               7.4             0.700         0.00             1.9      0.076
1               7.8             0.880         0.00             2.6      0.098
2               7.8             0.760         0.04             2.3      0.092
3              11.2             0.280         0.56             1.9      0.075
4               7.4             0.700         0.00             1.9      0.076
…               …                 …            …               …         …
1594            6.2             0.600         0.08             2.0      0.090
1595            5.9             0.550         0.10             2.2      0.062
1596            6.3             0.510         0.13             2.3      0.076
1597            5.9             0.645         0.12             2.0      0.075
1598            6.0             0.310         0.47             3.6      0.067

      free sulfur dioxide  total sulfur dioxide  density    pH  sulphates  \
0                    11.0                  34.0  0.99780  3.51       0.56
1                    25.0                  67.0  0.99680  3.20       0.68
2                    15.0                  54.0  0.99700  3.26       0.65
```

```
3                    17.0                    60.0  0.99800  3.16       0.58
4                    11.0                    34.0  0.99780  3.51       0.56
…                   …                    …       …   …         …
1594                 32.0                    44.0  0.99490  3.45       0.58
1595                 39.0                    51.0  0.99512  3.52       0.76
1596                 29.0                    40.0  0.99574  3.42       0.75
1597                 32.0                    44.0  0.99547  3.57       0.71
1598                 18.0                    42.0  0.99549  3.39       0.66

      alcohol  quality
0         9.4        5
1         9.8        5
2         9.8        5
3         9.8        6
4         9.4        5
…        …       …
1594     10.5        5
1595     11.2        6
1596     11.0        6
1597     10.2        5
1598     11.0        6

[1599 rows x 12 columns]
```

[9]:
```
data.describe()
```

[9]:
```
       fixed acidity  volatile acidity  citric acid  residual sugar  \
count    1599.000000       1599.000000  1599.000000     1599.000000
mean        8.319637          0.527821     0.270976        2.538806
std         1.741096          0.179060     0.194801        1.409928
min         4.600000          0.120000     0.000000        0.900000
25%         7.100000          0.390000     0.090000        1.900000
50%         7.900000          0.520000     0.260000        2.200000
75%         9.200000          0.640000     0.420000        2.600000
max        15.900000          1.580000     1.000000       15.500000

         chlorides  free sulfur dioxide  total sulfur dioxide      density  \
count  1599.000000          1599.000000           1599.000000  1599.000000
mean      0.087467            15.874922             46.467792     0.996747
std       0.047065            10.460157             32.895324     0.001887
min       0.012000             1.000000              6.000000     0.990070
25%       0.070000             7.000000             22.000000     0.995600
50%       0.079000            14.000000             38.000000     0.996750
75%       0.090000            21.000000             62.000000     0.997835
max       0.611000            72.000000            289.000000     1.003690

                pH    sulphates      alcohol      quality
```

```
count  1599.000000  1599.000000  1599.000000  1599.000000
mean      3.311113     0.658149    10.422983     5.636023
std       0.154386     0.169507     1.065668     0.807569
min       2.740000     0.330000     8.400000     3.000000
25%       3.210000     0.550000     9.500000     5.000000
50%       3.310000     0.620000    10.200000     6.000000
75%       3.400000     0.730000    11.100000     6.000000
max       4.010000     2.000000    14.900000     8.000000
```

Q2 *A)*

[10]:
```
data=pd.read_csv(r"C:DATA\Data.csv")
```

[11]:
```
data
```

[11]:
```
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes
```

[12]:
```
import warnings
warnings.filterwarnings('ignore')
```

[13]:
```
data['Age'].fillna(data['Age'].mean(),inplace=True)
data['Salary'].fillna(data['Salary'].mean(), inplace=True)
print("\n Mean of Age")
print(data['Age'].mean())
print("\n Mean of Salary")
print(data['Salary'].mean())
print(" \n New data")
print(data)
```

```
 Mean of Age
38.77777777777778

 Mean of Salary
63777.777777777774

 New data
   Country         Age         Salary Purchased
```

```
0   France  44.000000  72000.000000        No
1    Spain  27.000000  48000.000000       Yes
2  Germany  30.000000  54000.000000        No
3    Spain  38.000000  61000.000000        No
4  Germany  40.000000  63777.777778       Yes
5   France  35.000000  58000.000000       Yes
6    Spain  38.777778  52000.000000        No
7   France  48.000000  79000.000000       Yes
8  Germany  50.000000  83000.000000        No
9   France  37.000000  67000.000000       Yes
```

B)

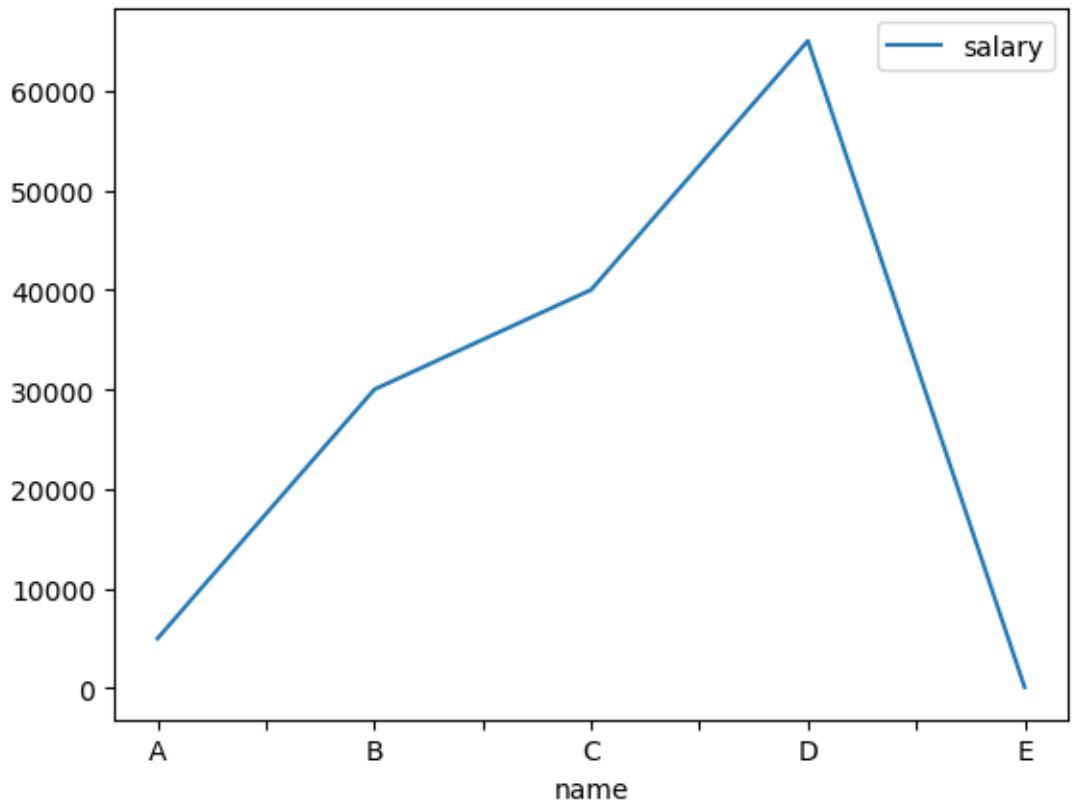
[14]:
```
new_data = {'name': ['A', 'B', 'C', 'D', 'E'],
            'salary': [5000,30000,40000,65000,100]}
```

[15]:
```
n=pd.DataFrame(new_data)
```

[16]:
```
n
```

[16]:
```
  name  salary
0    A    5000
1    B   30000
2    C   40000
3    D   65000
4    E     100
```

[17]:
```
n.plot('name','salary')
plt.show()
```



C)

[18]:
```
data=pd.read_csv(r"C:DATA\SOCR-HeightWeight.csv")
```

[19]:
```
print("First 10 rows:")
print(data.head(10))
```

```
First 10 rows:
     Height    Weight
0  65.78331  112.9925
1  71.51521  136.4873
2  69.39874  153.0269
3  68.21660  142.3354
4  67.78781  144.2971
5  68.69784  123.3024
6  69.80204  141.4947
7  70.01472  136.4623
8  67.90265  112.3723
9  66.78236  120.6672
```

[20]:
```
print("\nLast 10 rows:")
print(data.tail(10))
```

```
Last 10 rows:
         Height    Weight
24990  69.97767  125.3672
24991  71.91656  128.2840
24992  70.96218  146.1936
24993  66.19462  118.7974
24994  67.21126  127.6603
24995  69.50215  118.0312
24996  64.54826  120.1932
24997  64.69855  118.2655
24998  67.52918  132.2682
24999  68.87761  124.8742
```

[21]:
```
print("\n20 Random rows:")
print(data.sample(20))
```

```
20 Random rows:
         Height    Weight
9942   67.91700  123.5503
10644  67.74524  128.6228
5197   67.19993  111.4001
10437  64.35599  133.6729
19032  67.00578  120.8698
3037   66.03614  123.3208
19747  68.01268  132.8762
20730  67.35413  129.3491
21212  66.18245  118.4861
15796  69.78731  133.3097
14173  67.51440  139.3524
22557  67.62358  116.1185
1008   70.69674  135.8641
9744   69.29564  125.0373
8274   67.46519  122.2341
16348  68.15897  137.1696
10930  69.10888  139.3881
24291  65.89887  119.1833
14801  66.65826  124.0843
16881  65.90289  115.8894
```

[22]:
```
print("\nShape of the dataset:", data.shape)
```

```
Shape of the dataset: (25000, 2)
```

Q3 *A)*

[23]:
```
df = pd.read_csv(r"C:\Sudarshan\DATA\iris.csv")
```

[24]:
```
df
```

[24]:
```
     Sepal.Length  Sepal.Width  Petal.Length  Petal.Width    Species
0             5.1          3.5           1.4          0.2     setosa
1             4.9          3.0           1.4          0.2     setosa
2             4.7          3.2           1.3          0.2     setosa
3             4.6          3.1           1.5          0.2     setosa
4             5.0          3.6           1.4          0.2     setosa
..            …          …           …           …         …
145           6.7          3.0           5.2          2.3  virginica
146           6.3          2.5           5.0          1.9  virginica
147           6.5          3.0           5.2          2.0  virginica
148           6.2          3.4           5.4          2.3  virginica
149           5.9          3.0           5.1          1.8  virginica

[150 rows x 5 columns]
```

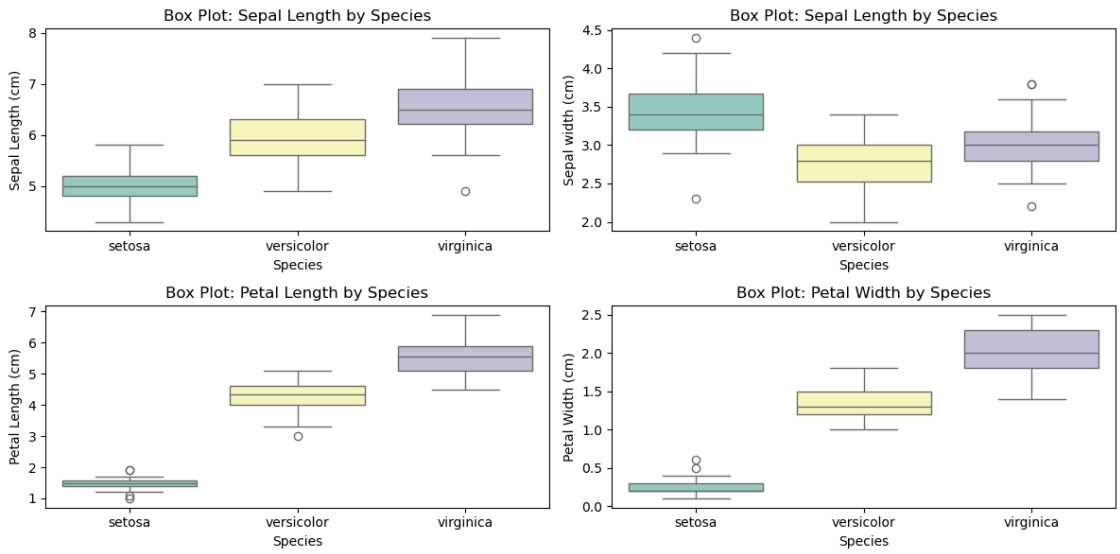
[25]:
```
plt.figure(figsize=(12, 6))
plt.subplot(2, 2, 1)
sns.boxplot(x='Species', y='Sepal.Length', data=df, palette='Set3')
plt.xlabel('Species')
plt.ylabel('Sepal Length (cm)')
plt.title('Box Plot: Sepal Length by Species')

plt.subplot(2, 2, 2)
sns.boxplot(x='Species', y='Sepal.Width', data=df, palette='Set3')
plt.xlabel('Species')
plt.ylabel('Sepal width (cm)')
plt.title('Box Plot: Sepal Length by Species')

plt.subplot(2, 2, 3)
sns.boxplot(x='Species', y='Petal.Length', data=df, palette='Set3')
plt.xlabel('Species')
plt.ylabel('Petal Length (cm)')
plt.title('Box Plot: Petal Length by Species')

plt.subplot(2, 2, 4)
sns.boxplot(x='Species', y='Petal.Width', data=df, palette='Set3')
plt.xlabel('Species')
plt.ylabel('Petal Width (cm)')
plt.title('Box Plot: Petal Width by Species')

plt.tight_layout()
plt.show()
```



B)

[26]:
```
data=pd.read_csv(r"C:DATA\SOCR-HeightWeight.csv")
```

[27]:
```
data
```

[27]:
```
         Height    Weight
0      65.78331  112.9925
1      71.51521  136.4873
2      69.39874  153.0269
3      68.21660  142.3354
4      67.78781  144.2971
…           …         …
24995  69.50215  118.0312
24996  64.54826  120.1932
24997  64.69855  118.2655
24998  67.52918  132.2682
24999  68.87761  124.8742

[25000 rows x 2 columns]
```

[28]:
```
data.describe()
```

[28]:
```
             Height        Weight
count  25000.000000  25000.000000
mean      67.993114    127.079421
std        1.901679     11.660898
min       60.278360     78.014760
25%       66.704397    119.308675
```

```
50%        67.995700    127.157750
75%        69.272958    134.892850
max        75.152800    170.924000
```

Q4 *A)*

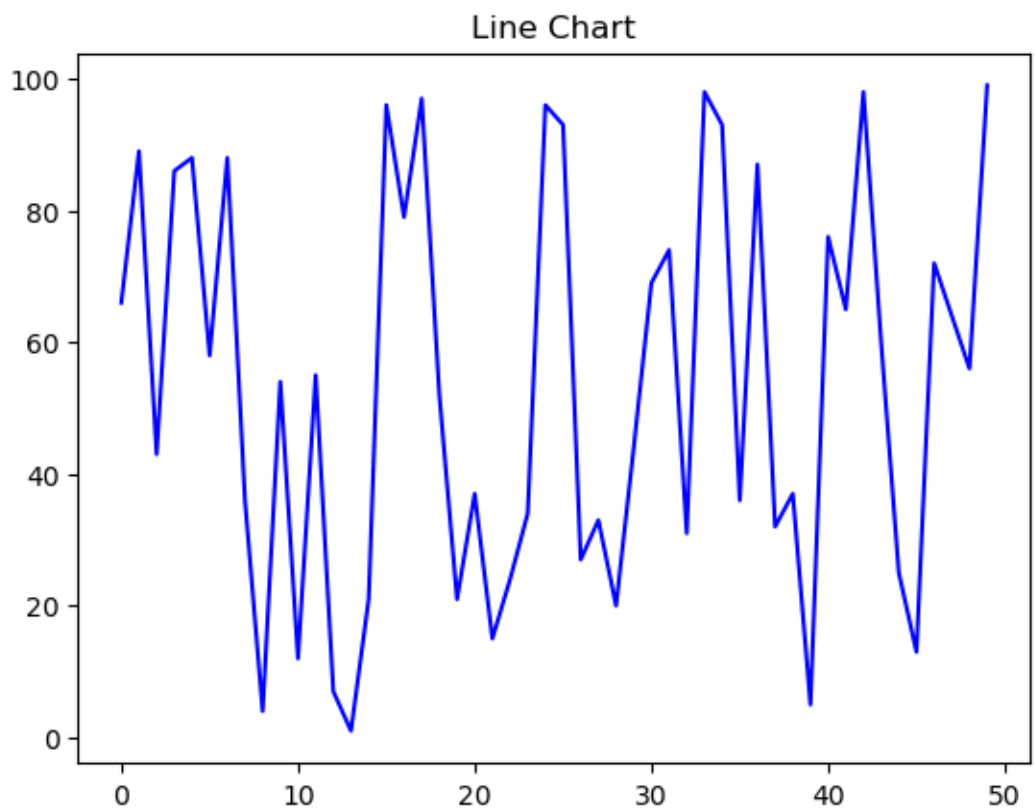
[29]:
```
data = np.random.randint(1, 100, 50)
```

[30]:
```
data
```

[30]:
```
array([66, 89, 43, 86, 88, 58, 88, 36,  4, 54, 12, 55,  7,  1, 21, 96, 79,
       97, 52, 21, 37, 15, 24, 34, 96, 93, 27, 33, 20, 44, 69, 74, 31, 98,
       93, 36, 87, 32, 37,  5, 76, 65, 98, 60, 25, 13, 72, 64, 56, 99],
      dtype=int32)
```

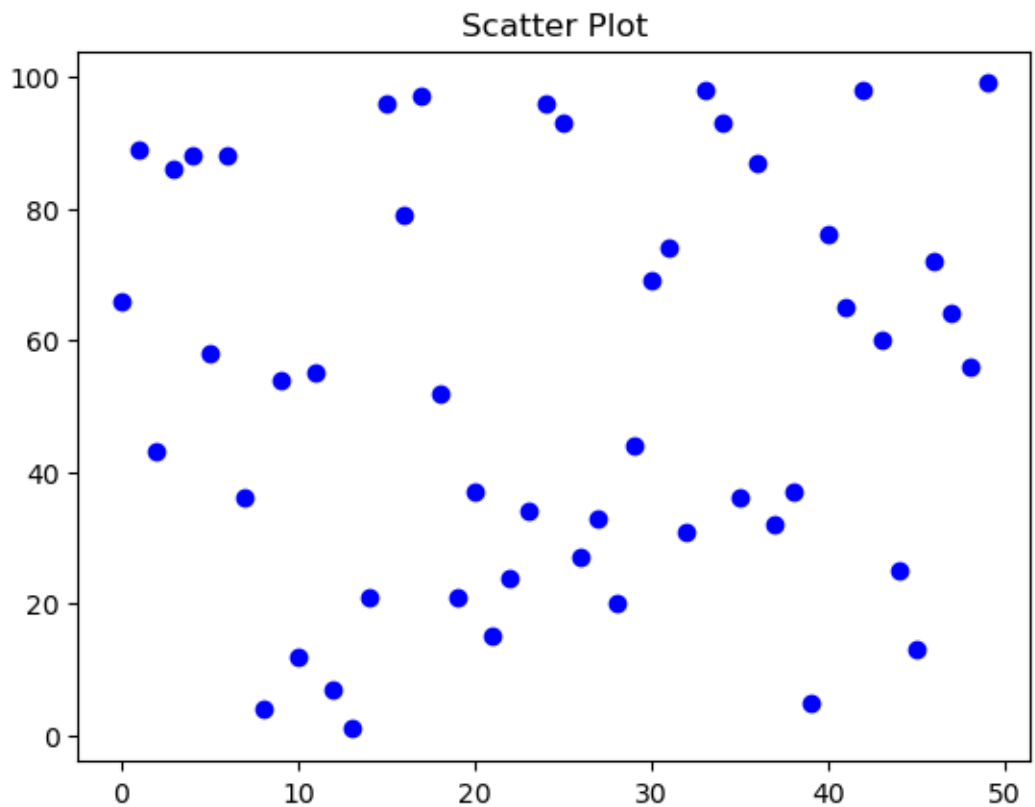
[31]:
```
plt.plot(data, color='blue')
plt.title('Line Chart')
```

[31]:
```
Text(0.5, 1.0, 'Line Chart')
```



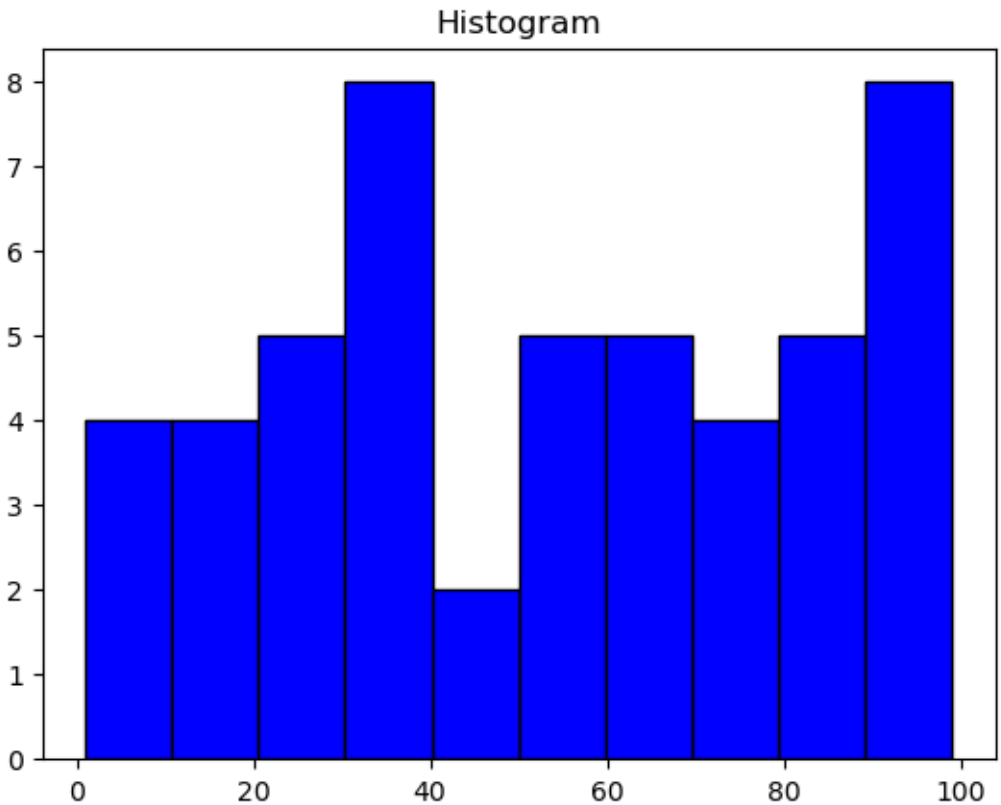
[32]:
```
plt.scatter(range(len(data)), data, color='blue')
plt.title('Scatter Plot')
```

[32]: Text(0.5, 1.0, 'Scatter Plot')

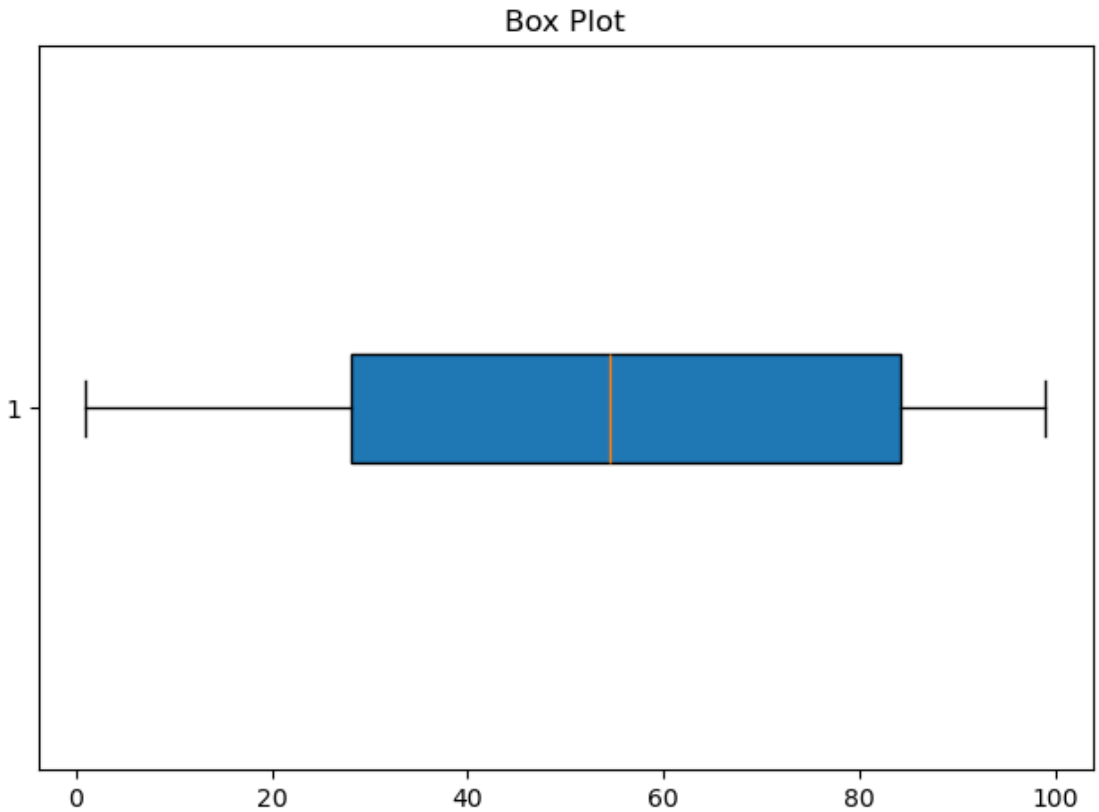


[33]:
```
plt.hist(data, bins=10, color='blue', edgecolor='black')
plt.title('Histogram')
```

[33]: Text(0.5, 1.0, 'Histogram')



[34]:
```
plt.boxplot(data, vert=False, patch_artist=True)
plt.title('Box Plot')
plt.tight_layout()
```



B)

[35]:
```
data=pd.read_csv(r"C:DATA\User_data.csv")
```

[36]:
```
data
```

[36]:
```
       User ID  Gender  Age  EstimatedSalary  Purchased
0     15624510    Male   19            19000          0
1     15810944    Male   35            20000          0
2     15668575  Female   26            43000          0
3     15603246  Female   27            57000          0
4     15804002    Male   19            76000          0
..         …     …   …               …         …
395   15691863  Female   46            41000          1
396   15706071    Male   51            23000          1
397   15654296  Female   50            20000          1
398   15755018    Male   36            33000          0
399   15594041  Female   49            36000          1

[400 rows x 5 columns]
```

[37]:
```
data.shape
```

[37]:
```
(400, 5)
```

[38]:
```
print("Number of rows and colums",data.shape)
```

```
Number of rows and colums (400, 5)
```

[39]:
```
data.dtypes
```

[39]:
```
User ID             int64
Gender             object
Age                 int64
EstimatedSalary     int64
Purchased           int64
dtype: object
```

[40]:
```
data.columns
```

[40]:
```
Index(['User ID', 'Gender', 'Age', 'EstimatedSalary', 'Purchased'],
dtype='object')
```

[41]:
```
data.describe
```

[41]:
```
<bound method NDFrame.describe of       User ID  Gender  Age  EstimatedSalary
Purchased
0    15624510    Male   19            19000          0
1    15810944    Male   35            20000          0
2    15668575  Female   26            43000          0
3    15603246  Female   27            57000          0
4    15804002    Male   19            76000          0
..        …     …  …              …        …
395  15691863  Female   46            41000          1
396  15706071    Male   51            23000          1
397  15654296  Female   50            20000          1
398  15755018    Male   36            33000          0
399  15594041  Female   49            36000          1

[400 rows x 5 columns]>
```

Q5 *A)*

[42]:
```
data=pd.read_csv(r"C:DATA\Data.csv")
```

[43]:
```
data
```

[43]:
```
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
```

```
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes
```

[44]:
```
data['Age'].fillna(data['Age'].mean(),inplace=True)
data['Salary'].fillna(data['Salary'].mean(), inplace=True)
print("\n Mean of Age")
print(data['Age'].mean())
print("\n Mean of Salary")
print(data['Salary'].mean())
print(" \n New data")
print(data)
```

```
 Mean of Age
38.77777777777778

 Mean of Salary
63777.777777777774

 New data
   Country        Age        Salary Purchased
0   France  44.000000  72000.000000        No
1    Spain  27.000000  48000.000000       Yes
2  Germany  30.000000  54000.000000        No
3    Spain  38.000000  61000.000000        No
4  Germany  40.000000  63777.777778       Yes
5   France  35.000000  58000.000000       Yes
6    Spain  38.777778  52000.000000        No
7   France  48.000000  79000.000000       Yes
8  Germany  50.000000  83000.000000        No
9   France  37.000000  67000.000000       Yes
```

B)

[45]:
```
data={'name': ['A', 'B', 'C', 'D'],
      'salary': [9000,98888,800000,20000]}
```

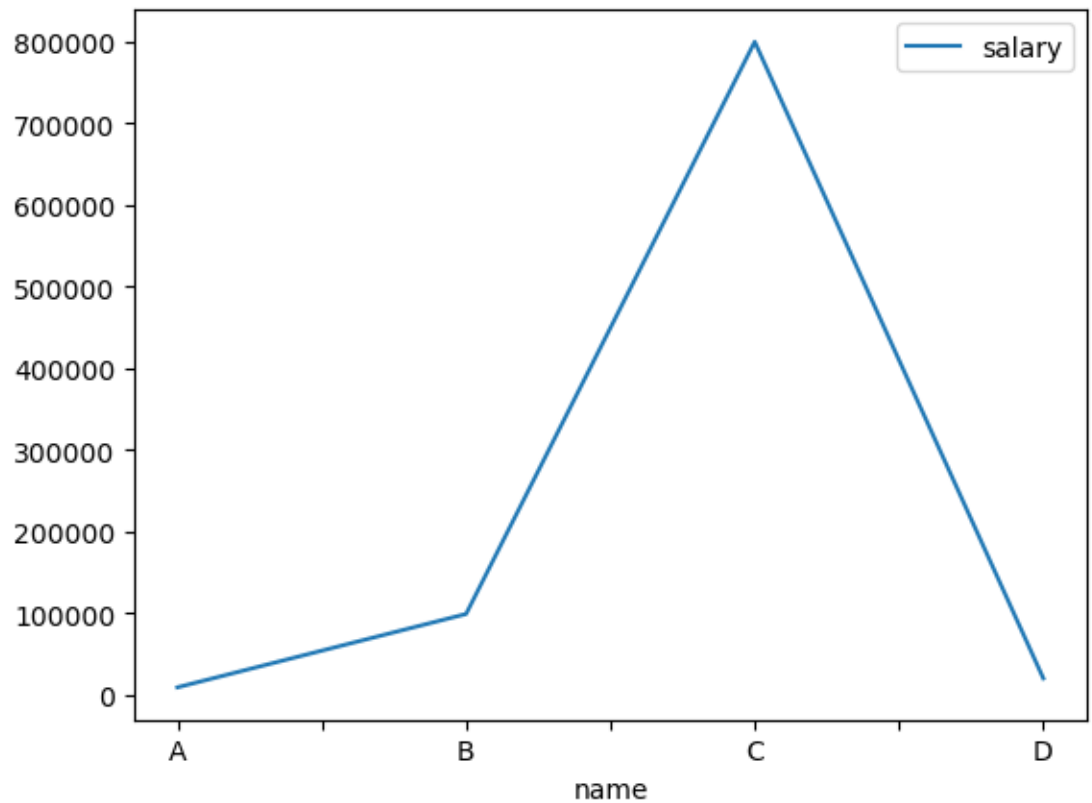
[46]:
```
df=pd.DataFrame(data)
```

[47]:
```
df
```

[47]:

	name	salary
0	A	9000
1	B	98888
2	C	800000
3	D	20000

[48]:
```
df.plot('name','salary')
plt.show()
```



C)

[49]:
```
HW=pd.read_csv(r"c:DATA\SOCR-HeightWeight.csv")
```

[50]:
```
HW.head(10)
```

[50]:

	Height	Weight
0	65.78331	112.9925
1	71.51521	136.4873
2	69.39874	153.0269
3	68.21660	142.3354
4	67.78781	144.2971

```
5  68.69784  123.3024
6  69.80204  141.4947
7  70.01472  136.4623
8  67.90265  112.3723
9  66.78236  120.6672
```

[51]:
```
HW.tail(10)
```

[51]:
```
          Height    Weight
24990  69.97767  125.3672
24991  71.91656  128.2840
24992  70.96218  146.1936
24993  66.19462  118.7974
24994  67.21126  127.6603
24995  69.50215  118.0312
24996  64.54826  120.1932
24997  64.69855  118.2655
24998  67.52918  132.2682
24999  68.87761  124.8742
```

[52]:
```
HW.sample(20)
```

[52]:
```
          Height    Weight
16277  66.30858  132.2852
23195  65.10427  113.6349
11381  67.69783  120.3788
10157  70.61416  127.9027
14190  66.38916  121.0049
20006  69.43118  133.4540
4612   67.71159  128.9420
21658  65.55386  118.7886
20792  68.72131  139.5527
22654  69.84827  129.8123
9287   68.96647  136.0886
15277  67.83322  107.5648
13133  70.25639  133.0404
19076  66.10658  124.0844
23737  67.42308  127.9037
15622  66.43466  128.8558
19024  68.71802  128.9584
12324  65.90593  108.2655
16917  70.21724  141.7782
15987  67.29300  137.0363
```

Q6

[53]:
```
from sklearn import preprocessing
```

[54]:
```
from sklearn.preprocessing import StandardScaler
```

[55]:
```
data=pd.read_csv(r"C:DATA\winequality-red.csv")
```

[56]:
```
data
```

[56]:
```
      fixed acidity  volatile acidity  citric acid  residual sugar  chlorides  \
0               7.4             0.700         0.00             1.9      0.076
1               7.8             0.880         0.00             2.6      0.098
2               7.8             0.760         0.04             2.3      0.092
3              11.2             0.280         0.56             1.9      0.075
4               7.4             0.700         0.00             1.9      0.076
…             …                 …           …              …         …
1594            6.2             0.600         0.08             2.0      0.090
1595            5.9             0.550         0.10             2.2      0.062
1596            6.3             0.510         0.13             2.3      0.076
1597            5.9             0.645         0.12             2.0      0.075
1598            6.0             0.310         0.47             3.6      0.067

      free sulfur dioxide  total sulfur dioxide  density    pH  sulphates  \
0                    11.0                  34.0  0.99780  3.51       0.56
1                    25.0                  67.0  0.99680  3.20       0.68
2                    15.0                  54.0  0.99700  3.26       0.65
3                    17.0                  60.0  0.99800  3.16       0.58
4                    11.0                  34.0  0.99780  3.51       0.56
…                   …                   …       …   …         …
1594                 32.0                  44.0  0.99490  3.45       0.58
1595                 39.0                  51.0  0.99512  3.52       0.76
1596                 29.0                  40.0  0.99574  3.42       0.75
1597                 32.0                  44.0  0.99547  3.57       0.71
1598                 18.0                  42.0  0.99549  3.39       0.66

      alcohol  quality
0         9.4        5
1         9.8        5
2         9.8        5
3         9.8        6
4         9.4        5
…       …       …
1594     10.5        5
1595     11.2        6
1596     11.0        6
1597     10.2        5
1598     11.0        6

[1599 rows x 12 columns]
```

[57]:
```
standard_scaler = StandardScaler()
standardized_data = standard_scaler.fit_transform(data)
```

[58]:
```
print(standardized_data)
```

```
[[-0.52835961  0.96187667 -1.39147228 … -0.57920652 -0.96024611
  -0.78782264]
 [-0.29854743  1.96744245 -1.39147228 …  0.1289504  -0.58477711
  -0.78782264]
 [-0.29854743  1.29706527 -1.18607043 … -0.04808883 -0.58477711
  -0.78782264]
 …
 [-1.1603431  -0.09955388 -0.72391627 …  0.54204194  0.54162988
   0.45084835]
 [-1.39015528  0.65462046 -0.77526673 …  0.30598963 -0.20930812
  -0.78782264]
 [-1.33270223 -1.21684919  1.02199944 …  0.01092425  0.54162988
   0.45084835]]
```

Q7 *A)*

[59]:
```
data=pd.read_csv(r"C:\Sudarshan\DATA\SOCR-HeightWeight.csv")
```

[60]:
```
data
```

[60]:

	Height	Weight
0	65.78331	112.9925
1	71.51521	136.4873
2	69.39874	153.0269
3	68.21660	142.3354
4	67.78781	144.2971
…	…	…
24995	69.50215	118.0312
24996	64.54826	120.1932
24997	64.69855	118.2655
24998	67.52918	132.2682
24999	68.87761	124.8742

[25000 rows x 2 columns]

[61]:
```
print(data['Height'].mean())
```

```
67.99311359679999
```

[62]:
```
print(data['Weight'].mean())
```

```
127.07942116080001
```

B)

[63]:
```python
import itertools

# Define a function to calculate the Manhattan distance between two points
def manhattan_distance(point1, point2):
    return abs(point1[0] - point2[0]) + abs(point1[1] - point2[1])

# Sample points as (x, y) coordinates
points = [(1, 2), (4, 5), (7, 8), (10, 11)]

# Initialize the sum of Manhattan distances to zero
total_manhattan_distance = 0

# Iterate through all unique pairs of points
point_pairs = list(itertools.combinations(points, 2))

for pair in point_pairs:
    distance = manhattan_distance(pair[0], pair[1])
    total_manhattan_distance += distance

# Print the sum of Manhattan distances
print("Sum of Manhattan distances between all pairs of points:",␣
 ↪total_manhattan_distance)
```

```
Sum of Manhattan distances between all pairs of points: 60
```

Q8 *A)*

[64]:
```python
data=pd.read_csv(r"C:DATA\iris.csv")
```

[65]:
```python
data
```

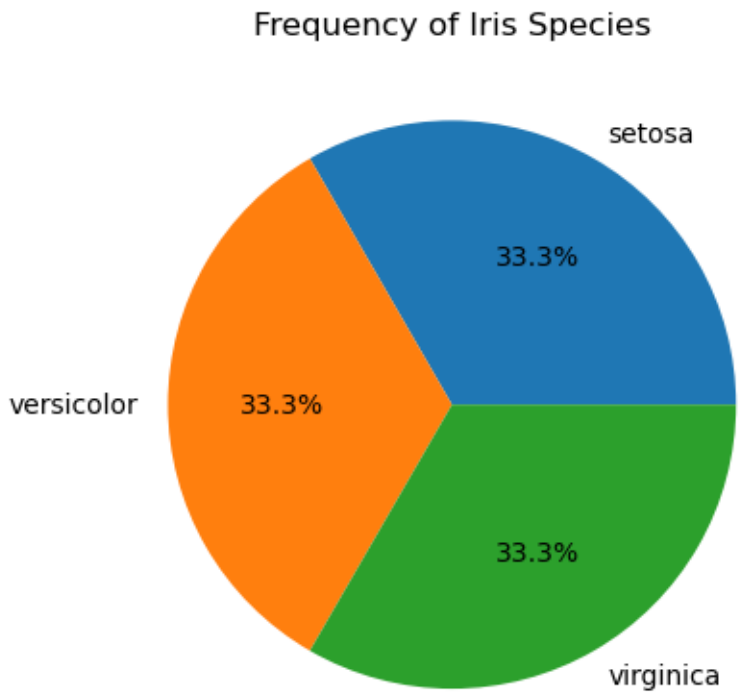
[65]:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
..	…	…	…	…	…
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

[150 rows x 5 columns]

[66]:
```python
s=data['Species'].value_counts()
```

[67]:
```
plt.pie(s,labels=s.index,autopct='%1.1f%%')
plt.title('Frequency of Iris Species')
plt.show()
```



B)

[68]:
```
data=pd.read_csv(r"C:DATA\winequality-red.csv")
```

[69]:
```
data
```

[69]:
```
      fixed acidity  volatile acidity  citric acid  residual sugar  chlorides  \
0               7.4             0.700         0.00             1.9      0.076
1               7.8             0.880         0.00             2.6      0.098
2               7.8             0.760         0.04             2.3      0.092
3              11.2             0.280         0.56             1.9      0.075
4               7.4             0.700         0.00             1.9      0.076
…               …                …            …               …         …
1594            6.2             0.600         0.08             2.0      0.090
1595            5.9             0.550         0.10             2.2      0.062
1596            6.3             0.510         0.13             2.3      0.076
1597            5.9             0.645         0.12             2.0      0.075
1598            6.0             0.310         0.47             3.6      0.067
```

```
      free sulfur dioxide  total sulfur dioxide  density    pH  sulphates  \
0                    11.0                  34.0  0.99780  3.51       0.56
1                    25.0                  67.0  0.99680  3.20       0.68
2                    15.0                  54.0  0.99700  3.26       0.65
3                    17.0                  60.0  0.99800  3.16       0.58
4                    11.0                  34.0  0.99780  3.51       0.56
…                     …                     …       …   …         …
1594                 32.0                  44.0  0.99490  3.45       0.58
1595                 39.0                  51.0  0.99512  3.52       0.76
1596                 29.0                  40.0  0.99574  3.42       0.75
1597                 32.0                  44.0  0.99547  3.57       0.71
1598                 18.0                  42.0  0.99549  3.39       0.66

      alcohol  quality
0         9.4        5
1         9.8        5
2         9.8        5
3         9.8        6
4         9.4        5
…       …      …
1594     10.5        5
1595     11.2        6
1596     11.0        6
1597     10.2        5
1598     11.0        6

[1599 rows x 12 columns]
```

[70]:
```
data.describe()
```

[70]:
```
       fixed acidity  volatile acidity  citric acid  residual sugar  \
count    1599.000000       1599.000000  1599.000000     1599.000000
mean        8.319637          0.527821     0.270976        2.538806
std         1.741096          0.179060     0.194801        1.409928
min         4.600000          0.120000     0.000000        0.900000
25%         7.100000          0.390000     0.090000        1.900000
50%         7.900000          0.520000     0.260000        2.200000
75%         9.200000          0.640000     0.420000        2.600000
max        15.900000          1.580000     1.000000       15.500000

         chlorides  free sulfur dioxide  total sulfur dioxide      density  \
count  1599.000000          1599.000000           1599.000000  1599.000000
mean      0.087467            15.874922             46.467792     0.996747
std       0.047065            10.460157             32.895324     0.001887
min       0.012000             1.000000              6.000000     0.990070
25%       0.070000             7.000000             22.000000     0.995600
50%       0.079000            14.000000             38.000000     0.996750
```

```
75%       0.090000            21.000000             62.000000     0.997835
max       0.611000            72.000000            289.000000     1.003690

                pH    sulphates      alcohol      quality
count  1599.000000  1599.000000  1599.000000  1599.000000
mean      3.311113     0.658149    10.422983     5.636023
std       0.154386     0.169507     1.065668     0.807569
min       2.740000     0.330000     8.400000     3.000000
25%       3.210000     0.550000     9.500000     5.000000
50%       3.310000     0.620000    10.200000     6.000000
75%       3.400000     0.730000    11.100000     6.000000
max       4.010000     2.000000    14.900000     8.000000
```

Q9 *A)*

[71]:
```
import numpy as np

# Define the original flattened array
original_array = np.array([[0, 1, 2],
                           [3, 4, 5],
                           [6, 7, 8]])

# Define the weights (one weight for each column)
weights = np.array([0.1, 0.2, 0.3])  # Adjust the weights as needed

# Calculate the weighted average along the specified axis (axis=1 for rows)
weighted_avg = np.average(original_array, axis=1, weights=weights)

# Print the results
print("Original flattened array:")
print(original_array)
print("\nWeighted average along the specified axis:")
print(weighted_avg)
```

```
Original flattened array:
[[0 1 2]
 [3 4 5]
 [6 7 8]]

Weighted average along the specified axis:
[1.33333333 4.33333333 7.33333333]
```

B)

[72]:
```
data=pd.read_csv(r"C:DATA\Advertising.csv")
```

[73]:
```
data
```

[73]:

```
        TV  Radio  Newspaper  Sales
0    230.1   37.8       69.2   22.1
1     44.5   39.3       45.1   10.4
2     17.2   45.9       69.3    9.3
3    151.5   41.3       58.5   18.5
4    180.8   10.8       58.4   12.9
..     …    …         …     …
195   38.2    3.7       13.8    7.6
196   94.2    4.9        8.1    9.7
197  177.0    9.3        6.4   12.8
198  283.6   42.0       66.2   25.5
199  232.1    8.6        8.7   13.4

[200 rows x 4 columns]
```

[74]:

```
data.describe()
```

[74]:

```
               TV       Radio   Newspaper       Sales
count  200.000000  200.000000  200.000000  200.000000
mean   147.042500   23.264000   30.554000   14.022500
std     85.854236   14.846809   21.778621    5.217457
min      0.700000    0.000000    0.300000    1.600000
25%     74.375000    9.975000   12.750000   10.375000
50%    149.750000   22.900000   25.750000   12.900000
75%    218.825000   36.525000   45.100000   17.400000
max    296.400000   49.600000  114.000000   27.000000
```

Q10 *A)*

[75]:

```
data = np.random.randint(1, 1000, 50)
```

[76]:

```
data
```

[76]:

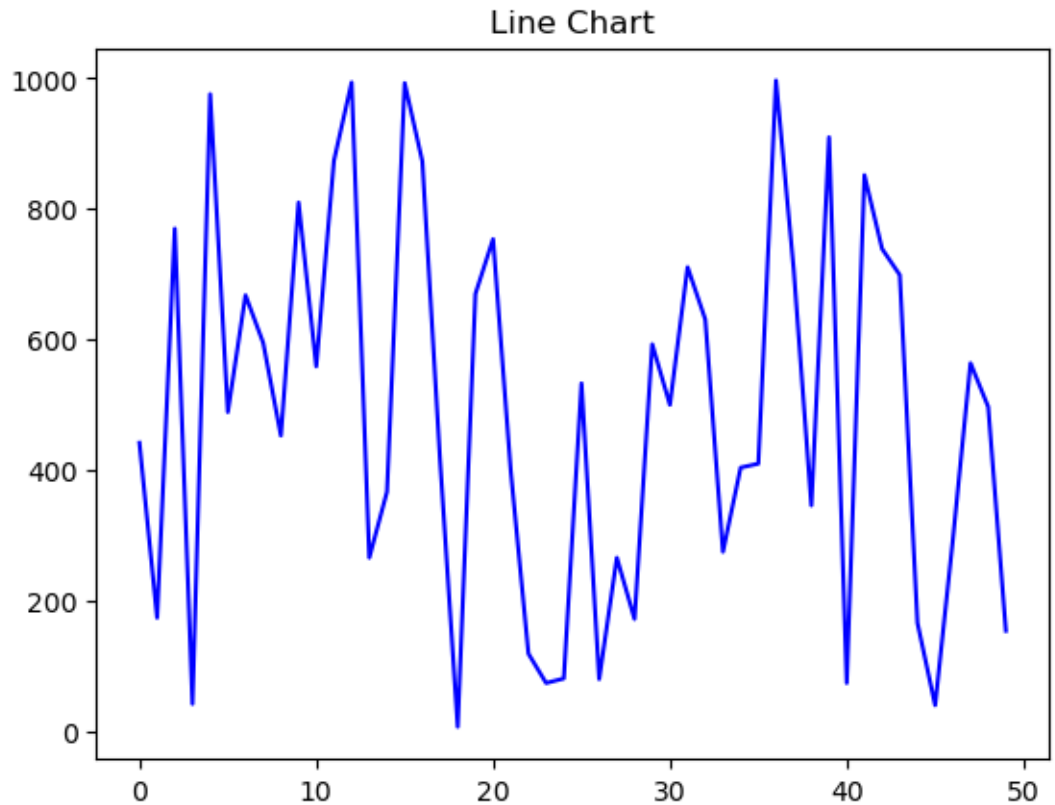
```
array([442, 174, 770,  42, 976, 489, 668, 595, 453, 810, 559, 874, 994,
       266, 367, 993, 873, 416,   7, 670, 754, 399, 119,  74,  81, 533,
        80, 266, 172, 593, 500, 711, 631, 275, 404, 410, 997, 708, 346,
       910,  74, 852, 739, 699, 166,  40, 293, 564, 497, 154], dtype=int32)
```

[77]:

```
plt.plot(data, color='blue')
plt.title('Line Chart')
```

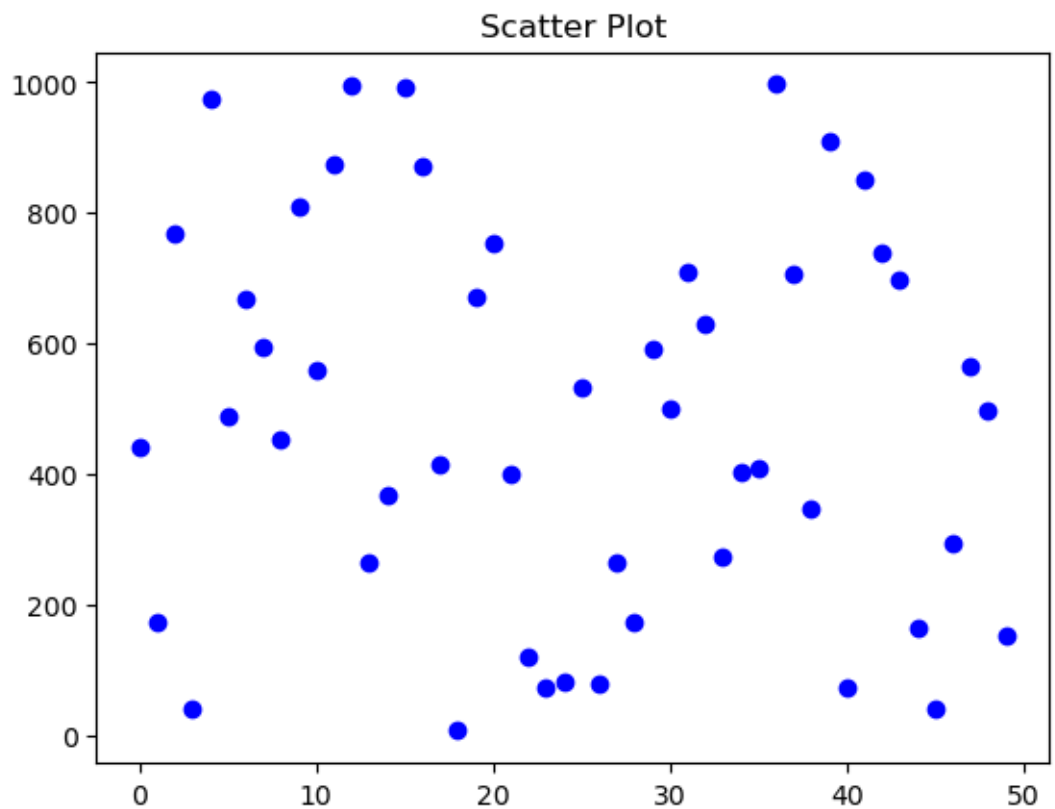
[77]:

```
Text(0.5, 1.0, 'Line Chart')
```



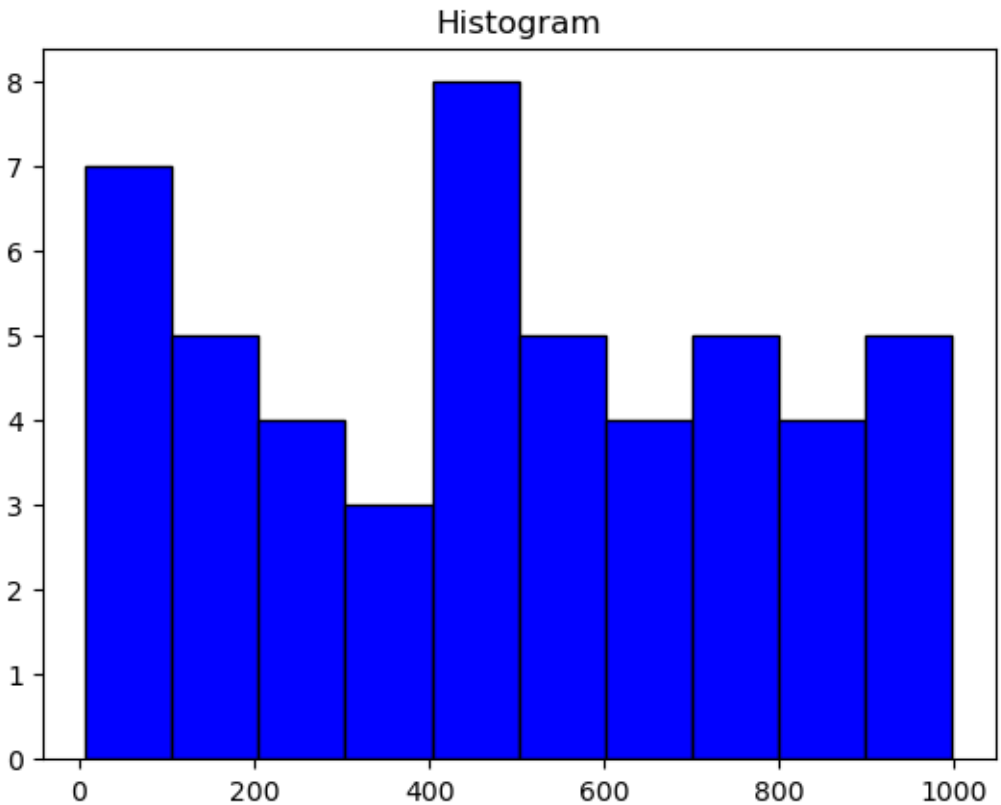
[78]:
```
plt.scatter(range(len(data)), data, color='blue')
plt.title('Scatter Plot')
```

[78]: Text(0.5, 1.0, 'Scatter Plot')

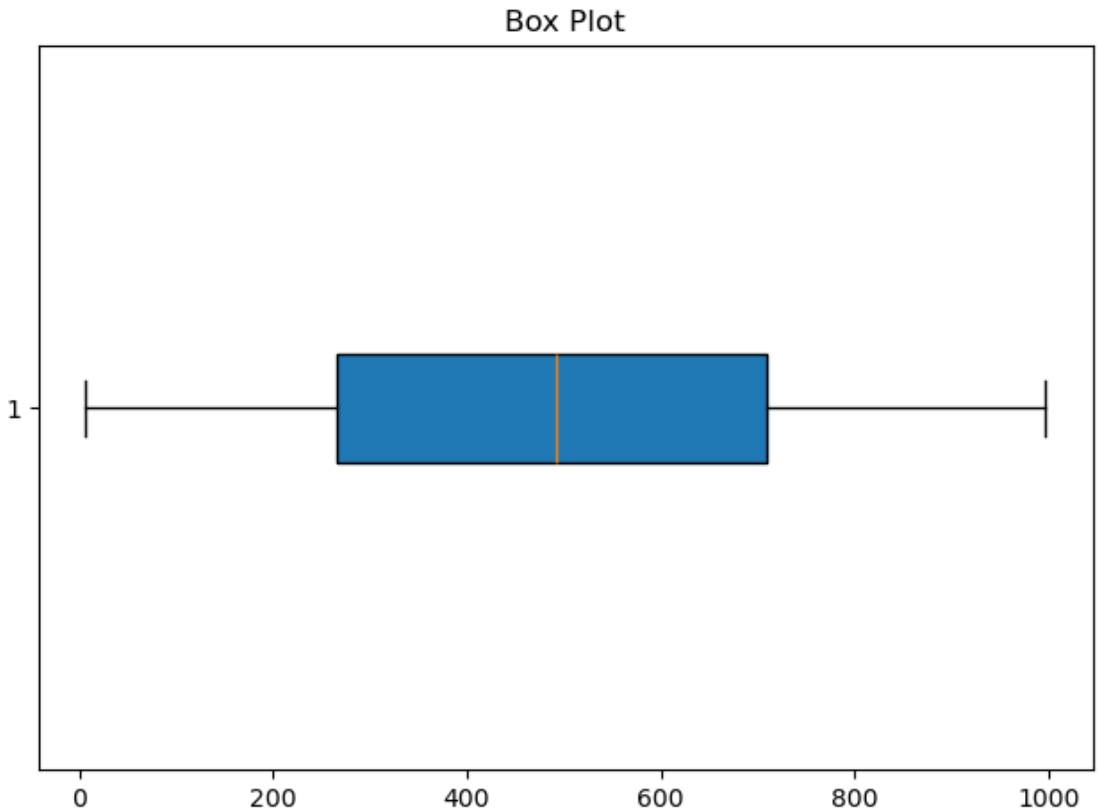


[79]:
```
plt.hist(data, bins=10, color='blue', edgecolor='black')
plt.title('Histogram')
```

[79]: Text(0.5, 1.0, 'Histogram')



[80]:
```
plt.boxplot(data, vert=False, patch_artist=True)
plt.title('Box Plot')
plt.tight_layout()
```



B)

[81]:
```
data=[['A',20],
      ['B',30],
      ['C',35],
      ['D',15]]
```

[82]:
```
cn=['names','marks']
```

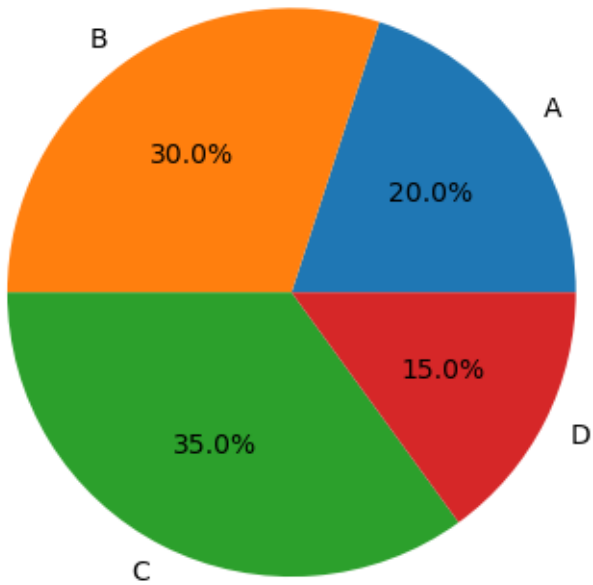
[83]:
```
df=pd.DataFrame(data,columns=cn)
```

[84]:
```
df
```

[84]:

	names	marks
0	A	20
1	B	30
2	C	35
3	D	15

[85]:
```
plt.pie(df['marks'],labels=df['names'],autopct='%1.1f%%')
plt.show()
```



Q11 *A)*

[86]:
```
df=pd.read_csv(r'C:DATA\iris.csv')
```

[87]:
```
df
```

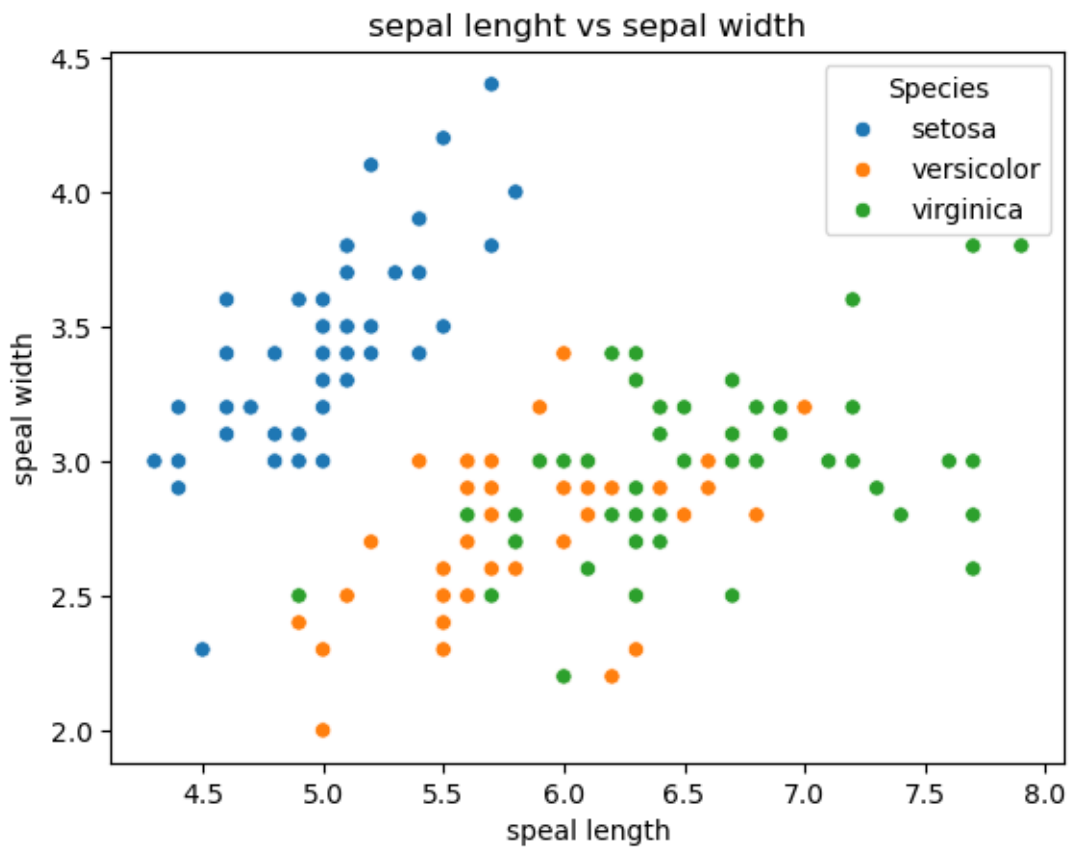
[87]:
```
     Sepal.Length  Sepal.Width  Petal.Length  Petal.Width    Species
0             5.1          3.5           1.4          0.2     setosa
1             4.9          3.0           1.4          0.2     setosa
2             4.7          3.2           1.3          0.2     setosa
3             4.6          3.1           1.5          0.2     setosa
4             5.0          3.6           1.4          0.2     setosa
..            …            …             …            …         …
145           6.7          3.0           5.2          2.3  virginica
146           6.3          2.5           5.0          1.9  virginica
147           6.5          3.0           5.2          2.0  virginica
148           6.2          3.4           5.4          2.3  virginica
149           5.9          3.0           5.1          1.8  virginica

[150 rows x 5 columns]
```

[88]:
```
sns.scatterplot(data=df,x='Sepal.Length',y='Sepal.Width',hue='Species')
plt.title('sepal lenght vs sepal width')
plt.xlabel('speal length')
```

```
plt.ylabel('speal width')
```

[88]: Text(0, 0.5, 'speal width')



B)

[89]:
```
data=[['raj',20,10000,'ele'],
      ['virat',30,5000,'civil'],
      ['rahul',35,99999,'ai'],
      ['sudarshan',30,330000,'webtecnology'],
      ['ritik',20,30000,'science'],
      ['nikhil',29,39999,'Datascience'],
      ['aaksh',35,455555,'computer'],
      ['unknown',25,76676,'macanical'],
      ['soham',80,900000,'entc'],
      ['viraj',55,60000,'xyz']]
```

[90]:
```
colname=['name','age','salary','department']
```

[91]:
```
data=pd.DataFrame(data,columns=colname)
```

[92]:
```
data
```

[92]:

	name	age	salary	department
0	raj	20	10000	ele
1	virat	30	5000	civil
2	rahul	35	99999	ai
3	sudarshan	30	330000	webtecnology
4	ritik	20	30000	science
5	nikhil	29	39999	Datascience
6	aaksh	35	455555	computer
7	unknown	25	76676	macanical
8	soham	80	900000	entc
9	viraj	55	60000	xyz

Q12 *A)*

[93]:
```
df=pd.read_csv(r'C:DATA\iris.csv')
```

[94]:
```
df
```

[94]:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
..	…	…	…	…	…
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

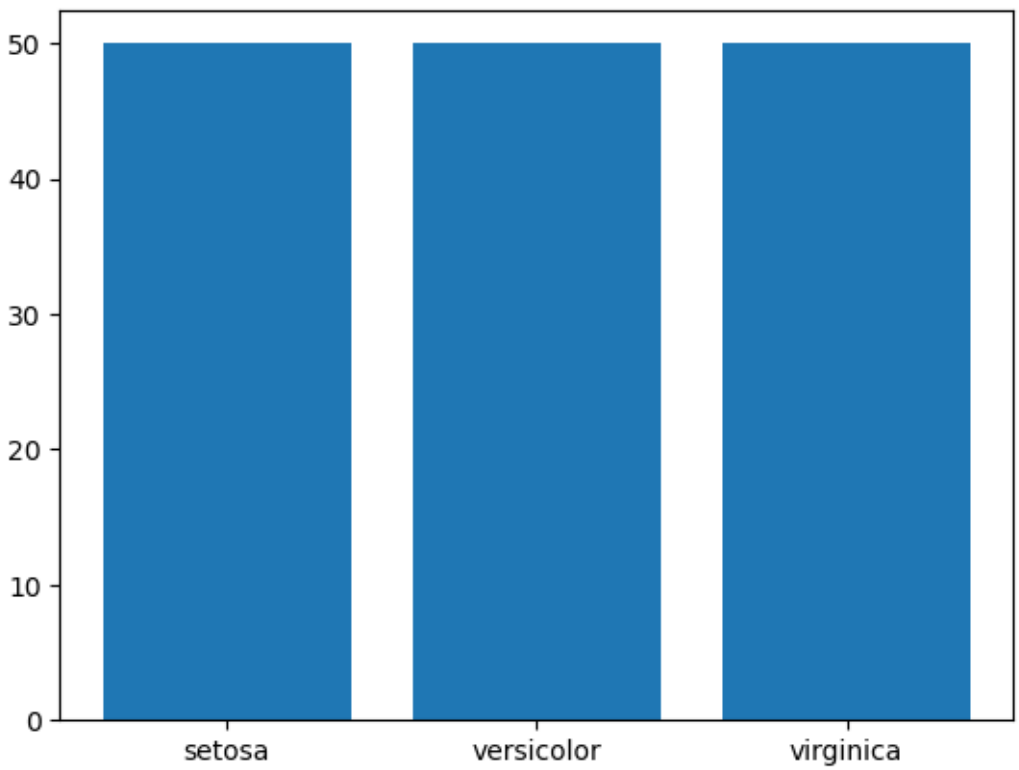
[150 rows x 5 columns]

[95]:
```
spe = df['Species'].value_counts()
```

[96]:
```
spe
```

[96]:
```
Species
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
```

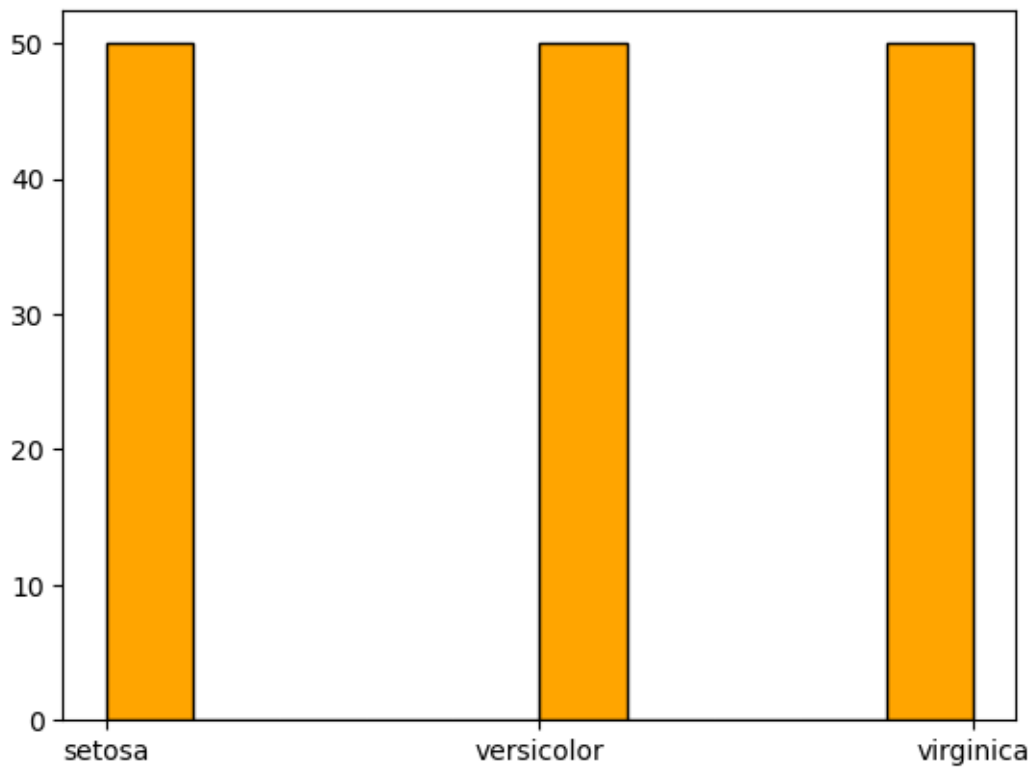
[97]:
```
plt.bar(spe.index,spe.values)
plt.show()
```



B)

[98]:
```
plt.hist(df['Species'], bins=10, color='orange', edgecolor='black')
```

[98]:
```
(array([50.,  0.,  0.,  0.,  0., 50.,  0.,  0.,  0., 50.]),
 array([0. , 0.2, 0.4, 0.6, 0.8, 1. , 1.2, 1.4, 1.6, 1.8, 2. ]),
 <BarContainer object of 10 artists>)
```



Q13

[99]:
```
data=pd.read_csv(r"C:DATA\winequality-red.csv")
```

[100]:
```
data
```

[100]:
```
      fixed acidity  volatile acidity  citric acid  residual sugar  chlorides  \
0               7.4             0.700         0.00             1.9      0.076
1               7.8             0.880         0.00             2.6      0.098
2               7.8             0.760         0.04             2.3      0.092
3              11.2             0.280         0.56             1.9      0.075
4               7.4             0.700         0.00             1.9      0.076
…               …                 …            …               …         …
1594            6.2             0.600         0.08             2.0      0.090
1595            5.9             0.550         0.10             2.2      0.062
1596            6.3             0.510         0.13             2.3      0.076
1597            5.9             0.645         0.12             2.0      0.075
1598            6.0             0.310         0.47             3.6      0.067

      free sulfur dioxide  total sulfur dioxide  density    pH  sulphates  \
0                    11.0                  34.0  0.99780  3.51       0.56
1                    25.0                  67.0  0.99680  3.20       0.68
```

```
2                    15.0                    54.0  0.99700  3.26       0.65
3                    17.0                    60.0  0.99800  3.16       0.58
4                    11.0                    34.0  0.99780  3.51       0.56
…                   …                     …       …   …         …
1594                 32.0                    44.0  0.99490  3.45       0.58
1595                 39.0                    51.0  0.99512  3.52       0.76
1596                 29.0                    40.0  0.99574  3.42       0.75
1597                 32.0                    44.0  0.99547  3.57       0.71
1598                 18.0                    42.0  0.99549  3.39       0.66

      alcohol  quality
0         9.4        5
1         9.8        5
2         9.8        5
3         9.8        6
4         9.4        5
…       …       …
1594     10.5        5
1595     11.2        6
1596     11.0        6
1597     10.2        5
1598     11.0        6

[1599 rows x 12 columns]
```

[110]:
```python
from sklearn.preprocessing import MinMaxScaler,␣
↪StandardScaler,Normalizer,Binarizer
```

[111]:
```python
# a. Rescaling
scaler = MinMaxScaler()
rescaled_data = scaler.fit_transform(data)
# b. Standardizing
standard_scaler = StandardScaler()
standardized_data = standard_scaler.fit_transform(data)
# c. Normalizing
normalizer = Normalizer()
normalized_data = normalizer.fit_transform(data)

print("Rescaled Data:\n", rescaled_data)
print("\nStandardized Data:\n", standardized_data)
print("\nNormalized Data:\n", normalized_data)
```

```
Rescaled Data:
 [[0.24778761 0.39726027 0.         … 0.13772455 0.15384615 0.4       ]
 [0.28318584 0.52054795 0.         … 0.20958084 0.21538462 0.4       ]
 [0.28318584 0.43835616 0.04       … 0.19161677 0.21538462 0.4       ]
 …
 [0.15044248 0.26712329 0.13       … 0.25149701 0.4        0.6       ]
```

```
 [0.11504425 0.35958904 0.12       … 0.22754491 0.27692308 0.4       ]
 [0.12389381 0.13013699 0.47       … 0.19760479 0.4        0.6       ]]

Standardized Data:
 [[-0.52835961  0.96187667 -1.39147228 … -0.57920652 -0.96024611
  -0.78782264]
 [-0.29854743  1.96744245 -1.39147228 …  0.1289504  -0.58477711
  -0.78782264]
 [-0.29854743  1.29706527 -1.18607043 … -0.04808883 -0.58477711
  -0.78782264]
 …
 [-1.1603431  -0.09955388 -0.72391627 …  0.54204194  0.54162988
   0.45084835]
 [-1.39015528  0.65462046 -0.77526673 …  0.30598963 -0.20930812
  -0.78782264]
 [-1.33270223 -1.21684919  1.02199944 …  0.01092425  0.54162988
   0.45084835]]

Normalized Data:
 [[0.19347777 0.01830195 0.         … 0.01464156 0.24576906 0.13072822]
 [0.10698874 0.01207052 0.         … 0.00932722 0.13442175 0.06858252]
 [0.13494887 0.01314886 0.00069205 … 0.01124574 0.16955114 0.08650569]
 …
 [0.1222319  0.00989496 0.00252225 … 0.01455142 0.21342078 0.11641133]
 [0.10524769 0.01150589 0.00214063 … 0.0126654  0.18195363 0.08919296]
 [0.12491328 0.00645385 0.00978487 … 0.01374046 0.22900768 0.12491328]]
```

[ ]:
```
**Q14**
```

[112]:
```
data=pd.read_csv(r"C:DATA\winequality-red.csv")
```

[113]:
```
data
```

[113]:
```
      fixed acidity  volatile acidity  citric acid  residual sugar  chlorides  \
0               7.4             0.700         0.00             1.9      0.076
1               7.8             0.880         0.00             2.6      0.098
2               7.8             0.760         0.04             2.3      0.092
3              11.2             0.280         0.56             1.9      0.075
4               7.4             0.700         0.00             1.9      0.076
…             …               …           …              …        …
1594            6.2             0.600         0.08             2.0      0.090
1595            5.9             0.550         0.10             2.2      0.062
1596            6.3             0.510         0.13             2.3      0.076
1597            5.9             0.645         0.12             2.0      0.075
1598            6.0             0.310         0.47             3.6      0.067

      free sulfur dioxide  total sulfur dioxide  density    pH  sulphates  \
0                    11.0                  34.0  0.99780  3.51       0.56
```

```
1                    25.0                    67.0  0.99680  3.20       0.68
2                    15.0                    54.0  0.99700  3.26       0.65
3                    17.0                    60.0  0.99800  3.16       0.58
4                    11.0                    34.0  0.99780  3.51       0.56
…                    …                    …       …   …        …
1594                 32.0                    44.0  0.99490  3.45       0.58
1595                 39.0                    51.0  0.99512  3.52       0.76
1596                 29.0                    40.0  0.99574  3.42       0.75
1597                 32.0                    44.0  0.99547  3.57       0.71
1598                 18.0                    42.0  0.99549  3.39       0.66

      alcohol  quality
0         9.4        5
1         9.8        5
2         9.8        5
3         9.8        6
4         9.4        5
…       …       …
1594     10.5        5
1595     11.2        6
1596     11.0        6
1597     10.2        5
1598     11.0        6

[1599 rows x 12 columns]
```

[114]:
```python
# a. Rescaling
scaler = MinMaxScaler()
rescaled_data = scaler.fit_transform(data)
# b. Standardizing
standard_scaler = StandardScaler()
standardized_data = standard_scaler.fit_transform(data)
# c. Binarizing
binarizer = Binarizer(threshold=0.5)
binarized_data = binarizer.fit_transform(data.drop(columns=['quality']))

print("Rescaled Data:\n", rescaled_data)
print("\nStandardized Data:\n", standardized_data)
print("\nBinarized Data:\n", binarized_data)
```

```
Rescaled Data:
 [[0.24778761 0.39726027 0.         … 0.13772455 0.15384615 0.4       ]
 [0.28318584 0.52054795 0.         … 0.20958084 0.21538462 0.4       ]
 [0.28318584 0.43835616 0.04       … 0.19161677 0.21538462 0.4       ]
 …
 [0.15044248 0.26712329 0.13       … 0.25149701 0.4        0.6       ]
 [0.11504425 0.35958904 0.12       … 0.22754491 0.27692308 0.4       ]
 [0.12389381 0.13013699 0.47       … 0.19760479 0.4        0.6       ]]
```

```
Standardized Data:
 [[-0.52835961  0.96187667 -1.39147228 … -0.57920652 -0.96024611
  -0.78782264]
 [-0.29854743  1.96744245 -1.39147228 …  0.1289504  -0.58477711
  -0.78782264]
 [-0.29854743  1.29706527 -1.18607043 … -0.04808883 -0.58477711
  -0.78782264]
 …
 [-1.1603431  -0.09955388 -0.72391627 …  0.54204194  0.54162988
   0.45084835]
 [-1.39015528  0.65462046 -0.77526673 …  0.30598963 -0.20930812
  -0.78782264]
 [-1.33270223 -1.21684919  1.02199944 …  0.01092425  0.54162988
   0.45084835]]

Binarized Data:
 [[1. 1. 0. … 1. 1. 1.]
 [1. 1. 0. … 1. 1. 1.]
 [1. 1. 0. … 1. 1. 1.]
 …
 [1. 1. 0. … 1. 1. 1.]
 [1. 1. 0. … 1. 1. 1.]
 [1. 0. 0. … 1. 1. 1.]]
```

Q15 *A)*

[115]:
```
df=pd.read_csv(r'C:DATA\iris.csv')
```

[116]:
```
df
```

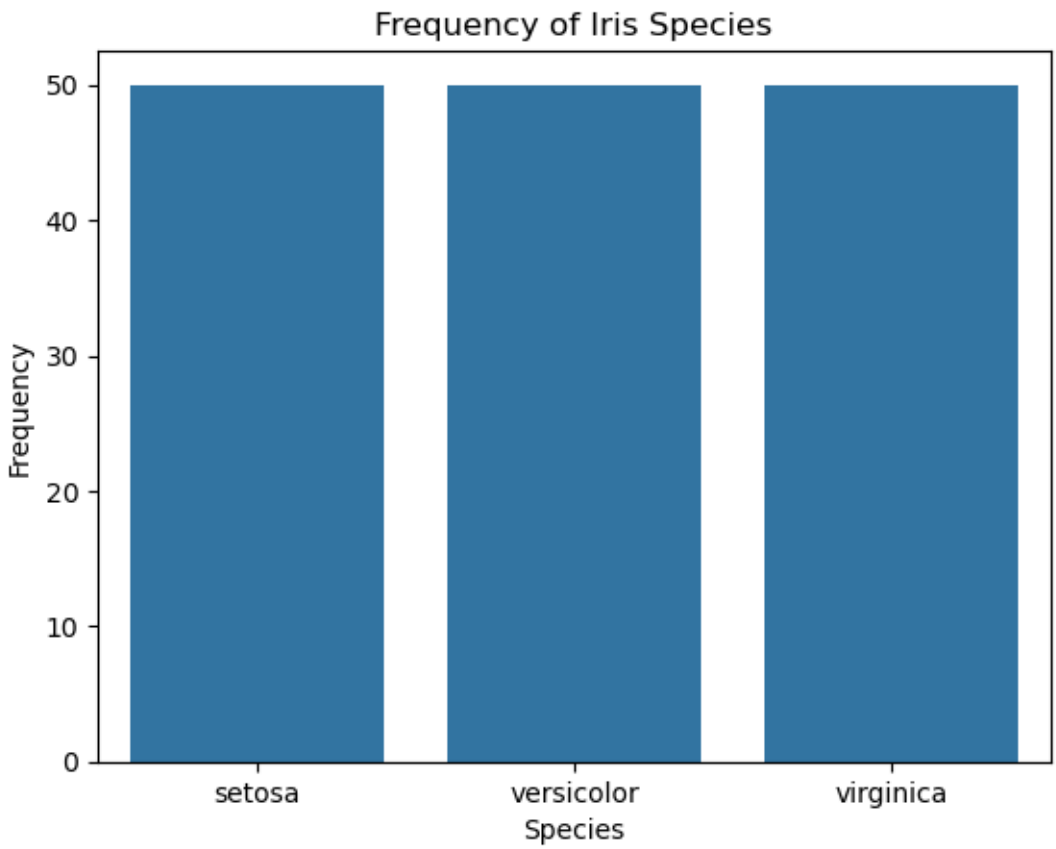
[116]:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
..	…	…	…	…	…
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

[150 rows x 5 columns]

[117]:
```
species_counts = df['Species'].value_counts()
sns.barplot(x=species_counts.index, y=species_counts.values)
```

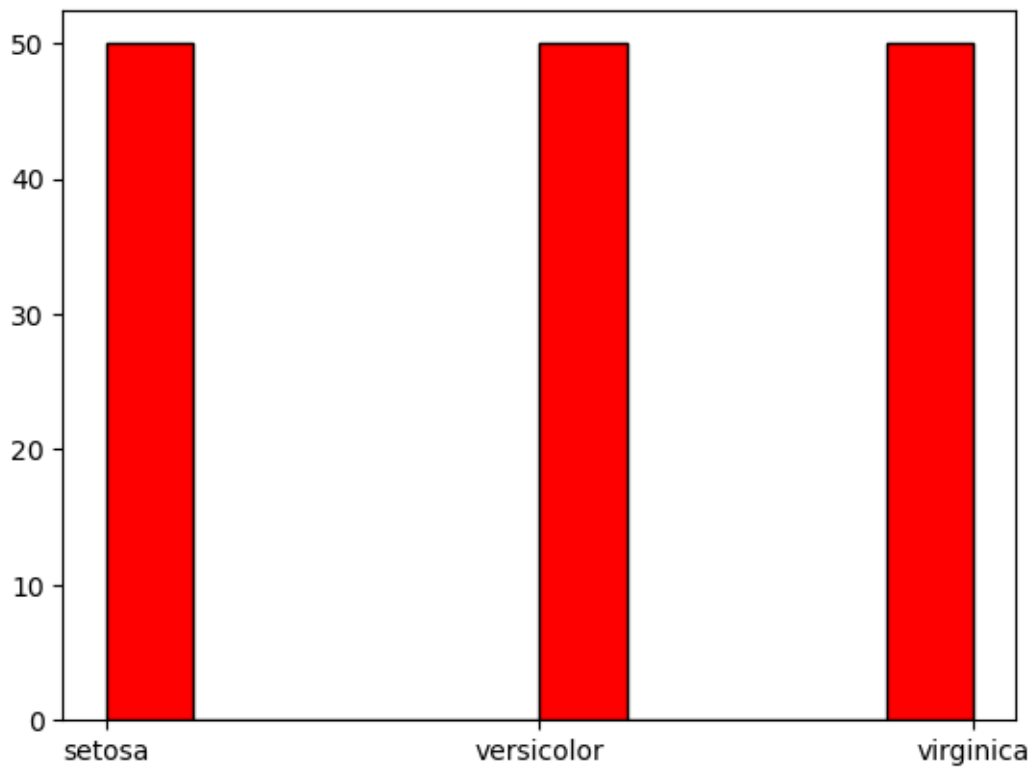
```
plt.xlabel('Species')
plt.ylabel('Frequency')
plt.title('Frequency of Iris Species')
plt.show()
```



B)

[118]:
```
plt.hist(df['Species'], bins=10, color='red', edgecolor='black')
```

[118]:
```
(array([50.,  0.,  0.,  0.,  0., 50.,  0.,  0.,  0., 50.]),
 array([0. , 0.2, 0.4, 0.6, 0.8, 1. , 1.2, 1.4, 1.6, 1.8, 2. ]),
 <BarContainer object of 10 artists>)
```



Q16 *A)*

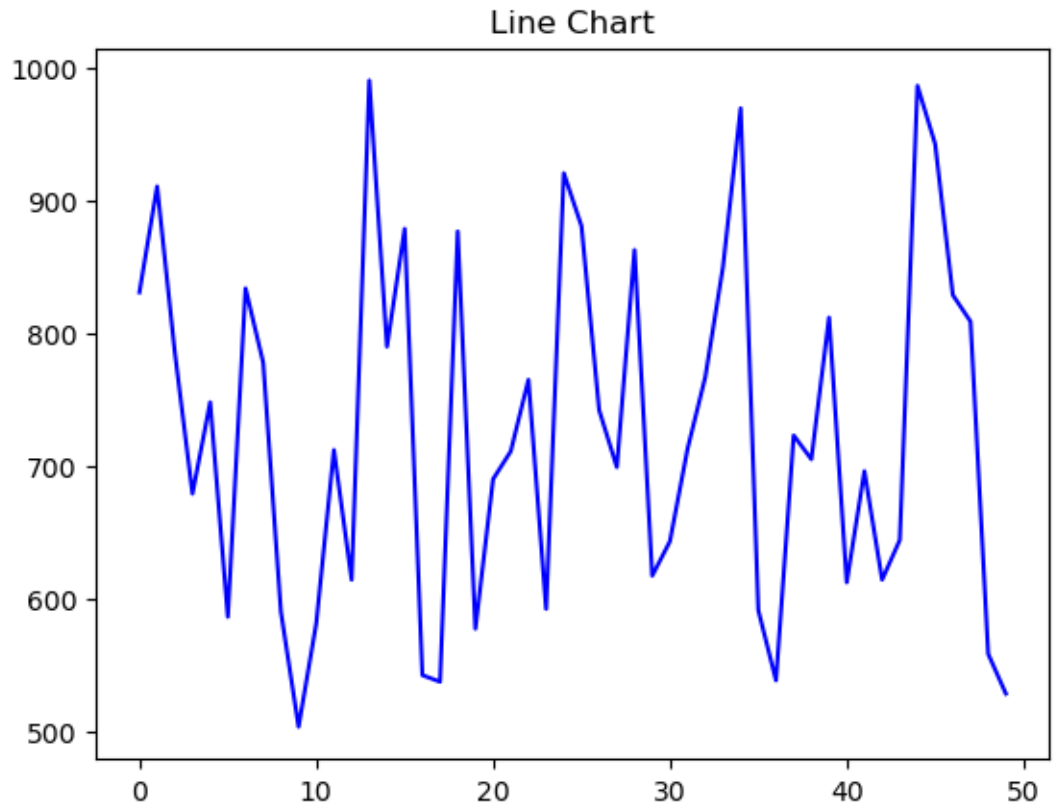
[119]:
```
data = np.random.randint(500, 1000, 50)
```

[120]:
```
data
```

[120]:
```
array([831, 911, 786, 679, 748, 586, 834, 778, 590, 503, 581, 712, 614,
       991, 790, 879, 542, 537, 877, 577, 690, 711, 765, 592, 921, 881,
       742, 699, 863, 617, 643, 713, 767, 850, 970, 591, 538, 723, 705,
       812, 612, 696, 614, 644, 987, 943, 829, 809, 558, 528], dtype=int32)
```

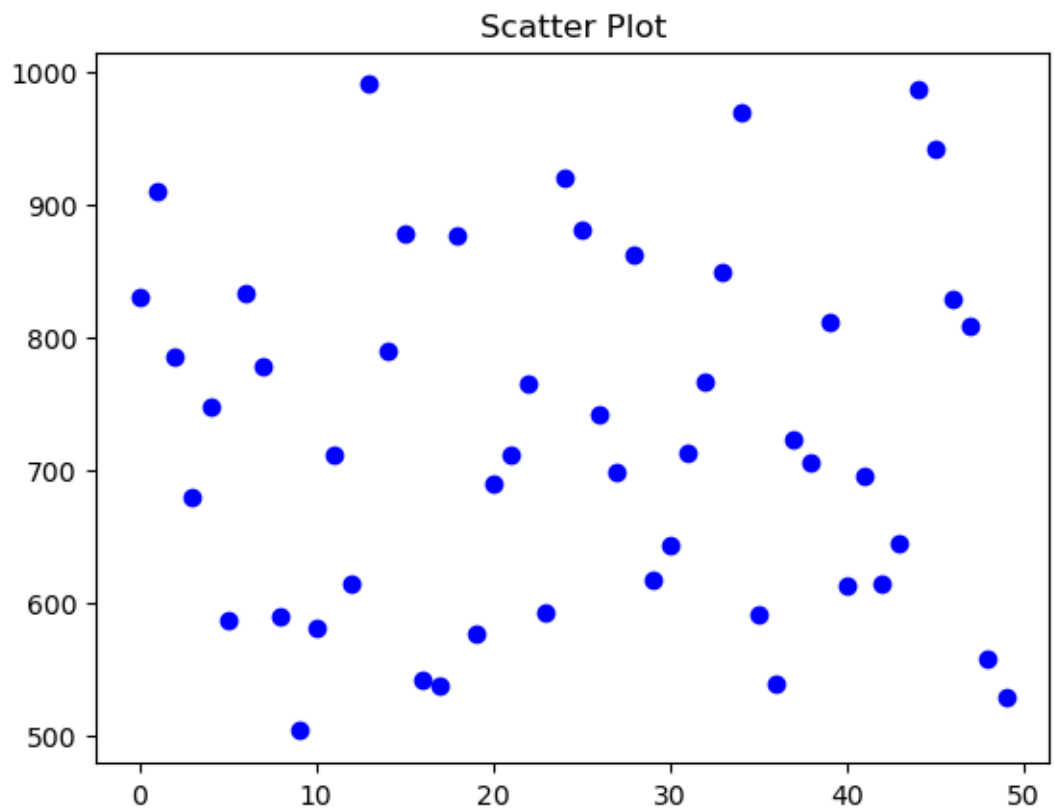
[121]:
```
plt.plot(data, color='blue')
plt.title('Line Chart')
```

[121]:
```
Text(0.5, 1.0, 'Line Chart')
```



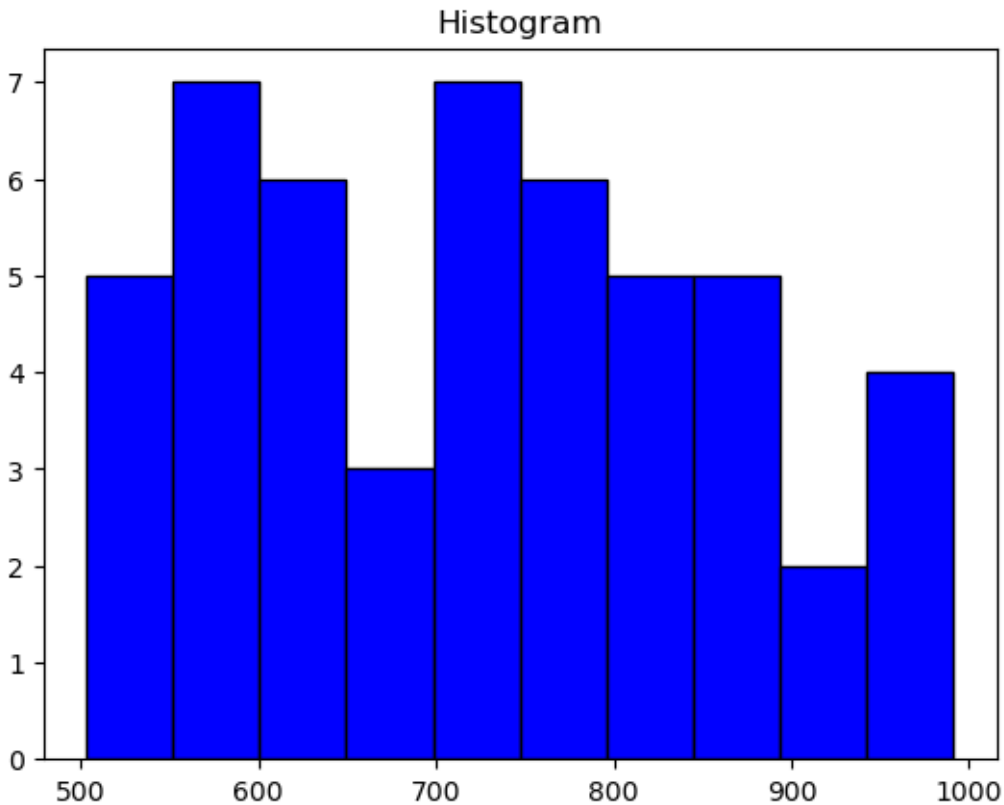
[122]:
```
plt.scatter(range(len(data)), data, color='blue')
plt.title('Scatter Plot')
```

[122]: Text(0.5, 1.0, 'Scatter Plot')

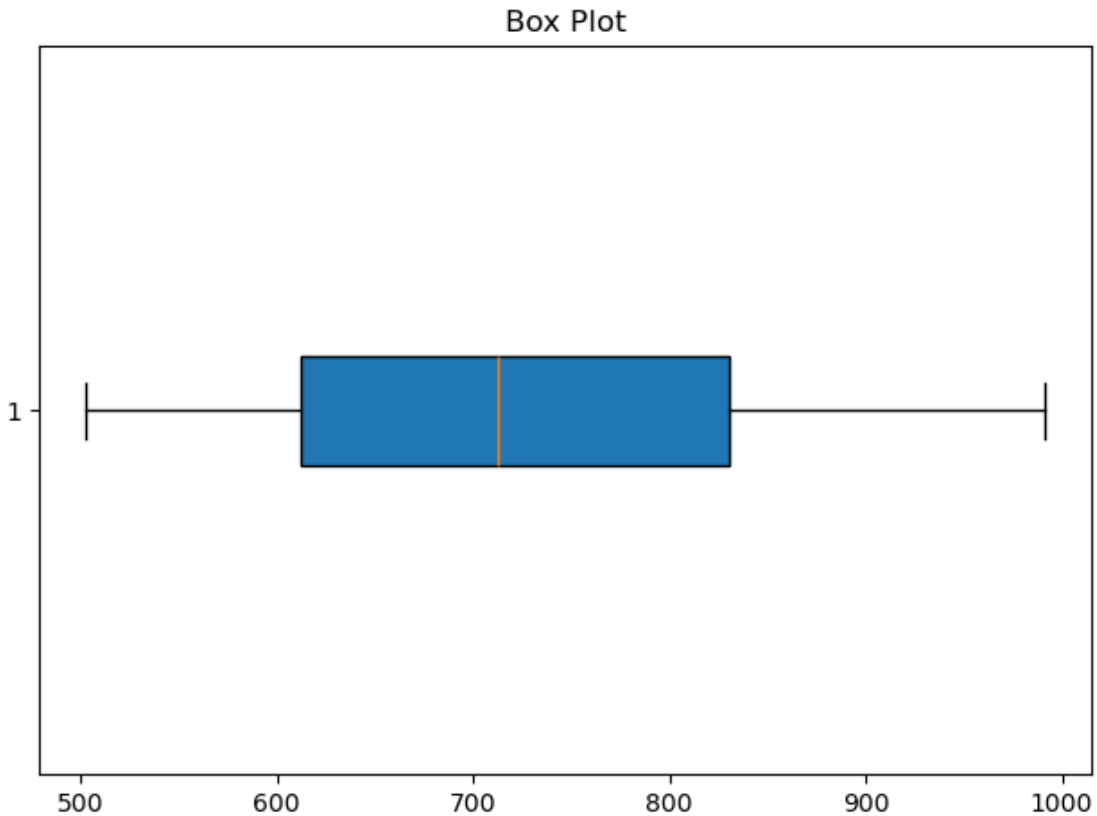


[123]:
```
plt.hist(data, bins=10, color='blue', edgecolor='black')
plt.title('Histogram')
```

[123]: Text(0.5, 1.0, 'Histogram')



[124]:
```
plt.boxplot(data, vert=False, patch_artist=True)
plt.title('Box Plot')
plt.tight_layout()
```



B)

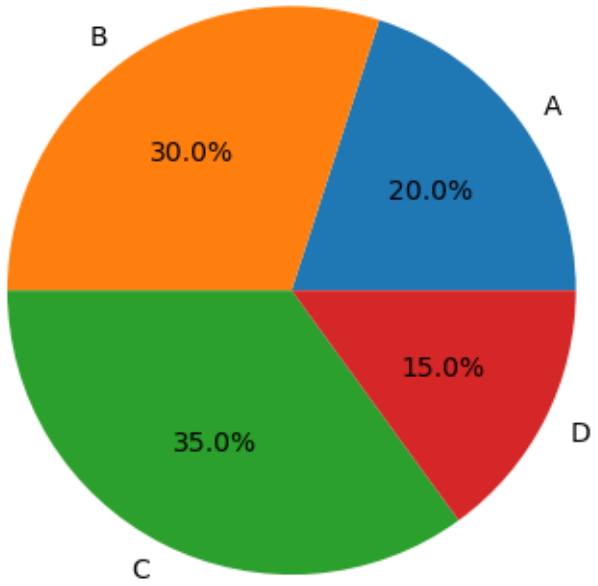
[125]:
```
data=[['A',20],
      ['B',30],
      ['C',35],
      ['D',15]]
cn=['names','marks']
df=pd.DataFrame(data,columns=cn)
```

[126]:
```
df
```

[126]:

	names	marks
0	A	20
1	B	30
2	C	35
3	D	15

[127]:
```
plt.pie(df['marks'],labels=df['names'],autopct='%1.1f%%')
plt.show()
```


B
A
30.0%
20.0%
15.0%
D
35.0%
C

```
[ ]:
```